ROSABON FINANCIAL SERVICES

Comprehensive QA & Testing Framework Strategy

A risk-based quality assurance strategy for the Rosabon App ecosystem

PREPARED BY	**Adebanjo Adeniji**
ROLE	Software Tester / QA Engineer
REVIEWED BY	Chief Technology Officer
ORGANIZATION	Rosabon Financial Services
VERSION	1.0
DATE	26 May 2026

Table of Contents

Executive Summary 3
1. Introduction 3
2. Fintech Industry Quality Requirements 5
3. Purpose and Scope of This Framework 6
4. Rosabon App Overview 7
5. Core Business Modules 7
6. QA Vision and Objectives 9
7. Risk-Based Quality Assurance Approach 10
8. Project Classification and Execution Matrix 11
9. Standard Test Timelines and SLAs 12
10. Quality Assurance Governance 13
11. Resourcing and Staffing Model 14
12. Testing Lifecycle Strategy 16
13. Integrated QA Workflow and CI/CD Pipeline 17
14. Functional Testing Strategy 18
15. Regression Testing Strategy 18
16. API Testing Strategy 18
17. Security Testing Strategy 19
18. Performance and Load Testing Strategy 19
19. Mobile Application Testing Strategy 19
20. Wallet System Testing Framework 20
21. Wealth Management Testing Framework 20
22. Investment Product Testing Framework 20
23. Loan Management Testing Framework 21
24. User Acceptance Testing (UAT) 21
25. Environment Management Strategy 22
26. Defect Management Process 23
27. Automation Testing Strategy 24
28. Project QA Execution Checklist 25
29. Release Readiness Framework 27
30. Reporting and Metrics Framework 28
31. Documentation Standards 29
32. Risk Management Framework 29
33. Compliance and Audit Readiness 30
34. QA Tooling Stack 31
35. Budget and Investment Plan 32
36. Implementation Roadmap 34
37. Immediate Action Plan 34
38. Conclusion 35

Executive Summary

The Rosabon App operates in one of the most sensitive segments of the digital economy. It holds customer funds, manages investment portfolios, processes live financial transactions, and will soon extend into lending. In this environment, a software defect is not a cosmetic inconvenience — it is a financial, regulatory, and reputational event.

This document defines a structured, risk-based Quality Assurance framework for the Rosabon App ecosystem. It moves QA from ad-hoc bug-finding toward a measurable, governed Quality Engineering discipline that protects financial accuracy, transaction integrity, and customer trust on every release.

At a glance, the framework delivers:

- **A risk classification model** that directs the heaviest testing effort to the highest-impact modules — wallet, authentication, investment returns, and loans.
- **Project Classification (Class A / B / C)** matching testing rigor to business impact, with published SLA timelines per class.
- **A documented resourcing model** (Section 11) showing the headcount required at current scale, pre-loan-launch, and at loan-launch.
- **Mandatory pre-launch gates for the loan module** (Section 23) requiring independent calculation validation, external security review, and Legal sign-off.
- **A baseline-first metrics framework** (Section 30) where Year-1 targets are ratified against the Phase-1 baseline measurement, not asserted in the abstract.
- **A named regulatory perimeter** (Section 33) covering CBN, NDPA/GAID and the NDPC, SEC under ISA 2025, and AML/NFIU obligations — with Legal co-sign required before external use.
- **A fully-loaded budget** (Section 35) of approximately **~~₦53 million~~ – ~~₦147 million~~ per year** depending on phase, covering people, tooling (capped), external pen-testing, performance infrastructure, and staging.

Implemented in full, this framework gives leadership documented confidence that every release has been validated systematically, securely, and reliably before it reaches a customer.

1. Introduction

Financial technology organizations operate in one of the most sensitive and high-risk segments of the digital economy. Unlike conventional software, a fintech application directly manages customer funds, financial records, investment portfolios, lending systems, live transactions, and highly sensitive personal data.

Because of this, a single software failure can cascade into consequences that extend far beyond the engineering team:

- Direct financial loss for customers or the business
- Erosion of customer trust and brand reputation
- Regulatory exposure and potential legal liability
- Security breaches and data compromise

- Operational disruption and service downtime

For these reasons, Quality Assurance in a fintech context is not a final checkpoint — it is a business-critical operational discipline. It is accountable for product reliability, financial accuracy, transaction integrity, security assurance, release confidence, and the continuous protection of customer trust.

This document establishes a comprehensive QA and Testing Framework Strategy for the **Rosabon App ecosystem**. It gives management, engineering leadership, and stakeholders documented assurance that every release, financial workflow, and customer interaction is validated systematically, securely, and reliably before production deployment.

2. Fintech Industry Quality Requirements

Fintech systems differ fundamentally from standard applications because they process customer financial assets, digital wallets, investment portfolios, payment workflows, identity verification, and sensitive user data. These demands translate into six non-negotiable quality dimensions that shape every decision in this framework.

Quality dimension	What it requires
Financial accuracy	Every calculation, balance update, interest computation, investment return, and repayment must be precise to the smallest unit and independently verifiable.
Transaction integrity	Transactions must complete reliably, prevent duplication, maintain accurate balances, fail safely, remain traceable, and support recovery.
Security assurance	The system must defend against unauthorized access, data leakage, session hijacking, fraudulent transactions, privilege escalation, and weak authentication.
Reliability & availability	Downtime and instability directly affect revenue, customer confidence, operations, and regulatory standing.
Compliance readiness	Audit trails, data traceability, transaction logging, access monitoring, and secure data handling must be maintained at all times.
Scalability	The platform must absorb rising transaction volumes, concurrent users, customer growth, and new business features without degradation.

Implication for QA: Because of these demands, QA operations at Rosabon must be structured, risk-focused, process-driven, automated where practical, measurable, and continuously monitored.

3. Purpose and Scope of This Framework

3.1 Purpose

This framework establishes a comprehensive quality assurance strategy that ensures the reliability of the Rosabon App ecosystem, reduces production defects, secures financial transactions and customer data, and builds genuine confidence in every production release. In doing so, it standardizes testing practices and strengthens collaboration between engineering, QA, and business teams.

3.2 What This Framework Governs

- Test planning, design, and execution
- Release validation and quality gates
- Defect management and triage
- Automation direction and tooling
- Project classification and SLA-based testing timelines
- Reporting standards and quality metrics
- Tooling investment and budget planning
- Risk mitigation and quality governance

4. Rosabon App Overview

The Rosabon App is a fintech mobile application that provides customers with a connected suite of digital financial services: wealth management, investment products, a digital wallet, financial transactions, and — in an upcoming release — loan management.

The application functions as a complete digital financial ecosystem in which customers fund wallets, enrol in investment products, monitor portfolios, and transact in real time. Given the financial sensitivity of these operations, the platform requires a mature and tightly structured quality assurance framework.

5. Core Business Modules

5.1 Wealth Management

Enables customers to enrol in investment products and manage their portfolios end to end.

- Investment onboarding and funding
- Portfolio monitoring and returns calculation
- Transaction history and maturity tracking
- Withdrawal and redemption operations

5.2 Investment Products

The platform offers three investment products — RWIN, REAP, and RTN — each with its own business rules, financial calculations, eligibility conditions, return structures, and transaction behaviours. These products require intensive functional and financial validation.

Product	Primary focus of validation
RWIN	Enrolment, funding validation, return calculations, eligibility checks, withdrawal behaviour
REAP	Investment creation, portfolio visibility, transaction validation, maturity handling
RTN	Product logic validation, financial computations, wallet interaction, user transaction handling

5.3 Wallet System

The wallet supports customer transactions and balance management, including funding, debit and credit operations, transfers, transaction logging, and notifications. **This is the highest-risk financial area in the platform and demands strict transaction-integrity testing.**

5.4 Loan Module (Upcoming)

The lending capability is under development for a future release. Expected capabilities include loan application, eligibility verification, automated underwriting, approval workflows, disbursement, repayment schedules, and repayment tracking. Because lending directly affects

financial operations and risk exposure, the module requires extensive validation and proactive QA planning ahead of launch.

6. QA Vision and Objectives

6.1 Vision

> *To establish a scalable, secure, reliable, and business-aligned quality assurance framework that protects customer trust, minimizes production risk, and supports operational excellence across the Rosabon App ecosystem.*

6.2 Objectives

- Minimize production defects and defect leakage
- Improve release confidence and defect detection before release
- Protect financial integrity and ensure stable customer experiences
- Standardize testing processes and strengthen cross-team collaboration
- Enable scalable automation adoption and support business continuity
- Strengthen management confidence in every release decision

7. Risk-Based Quality Assurance Approach

Not every part of the application carries the same business risk. Testing effort is therefore prioritized by a risk score derived from business impact and likelihood, ensuring the most sensitive systems receive the deepest coverage. The matrix below drives test depth, automation priority, and release-gate strictness.

Module / area	Risk level	Business impact
Wallet transactions	CRIT	Direct movement of customer funds
Authentication & access	CRIT	Account security and unauthorized access
Investment returns	CRIT	Financial accuracy of customer earnings
Loan calculations	CRIT	Financial and legal exposure
API integrations	HIGH	Service-to-service communication
Transaction history	HIGH	Customer trust and auditability
Notifications	MED	Customer communication
Profile management	MED	User account management
UI styling	LOW	Cosmetic / presentation only

How risk level drives effort: Critical areas receive exhaustive functional, edge-case, security, and regression coverage on every release and are the first candidates for automation. Low-risk areas are validated through smoke checks and periodic regression.

8. Project Classification and Execution Matrix

Risk-based prioritization (Section 7) tells us where to focus inside the product; project classification tells us how much rigor to apply to each individual change, feature, or deployment. Every upcoming initiative is classified into one of three classes, which automatically sets the testing scope and timeline SLA.

CLASS A	CLASS B	CLASS C
High / Fintech Impact	**Medium Impact**	**Low / Cosmetic Impact**
TRIGGERS • Core architecture changes • Transaction & wallet engines • Investment products (RWIN, REAP, RTN) • Payment / payout integrations • Loan underwriting parameters • Authentication mechanisms • High-risk ledger mutations	**TRIGGERS** • Feature enhancements (non-monetary) • Internal admin / portal updates • Push-notification logic • Standard API payload updates • Functional screen enhancements	**TRIGGERS** • Static copy revisions • CSS / UI fine-tuning • Tracking pixel additions • Localized low-risk bug fixes • Minor isolated patches
REQUIRED RIGOR • Complete E2E regression • Security penetration tests • API load / stress verification • Data-integrity schema assertions • Cross-platform mobile compliance • Formal UAT sign-off	**REQUIRED RIGOR** • Scoped functional validation • Targeted regression sweeps • API contract validation • Cross-device interface checks	**REQUIRED RIGOR** • Quick isolated smoke test • Targeted fix verification • Brief visual check (1 primary device)

Outcome: Each change is routed through testing rigor proportionate to its risk. A static copy update should not consume the same QA effort as a wallet-engine refactor, and a wallet-engine refactor should never reach production through the lighter Class C gate.

9. Standard Test Timelines and SLAs

Each project class carries a defined testing-timeline SLA that runs from requirements intake to production release-readiness validation. These SLAs let product, engineering, and leadership plan launch dates with realistic QA expectations rather than guesswork.

Testing stage	Class A — High	Class B — Medium	Class C — Low
Requirements review & test planning	2 – 3 days	1 day	0.5 day
Test case design & tooling configuration	3 – 5 days	1 – 2 days	0.5 day
Functional Test Run 1 & API verification	4 – 6 days	2 – 3 days	1 day
Defect triage, fix verification & Run 2	3 – 4 days	1 – 2 days	0.5 day
Full regression & cross-device suite	3 days	1 day	Smoke only
Security, performance & UAT sign-off	3 – 5 days	1 day	N/A
Release readiness gate validation	1 day	0.5 day	0.5 day
Total estimated testing duration	**19 – 27 days**	**7 – 12 days**	**2 – 3 days**

How to read this: the timeline is calendar time end-to-end, not effort hours. It assumes blockers are resolved promptly and the relevant environments are available; significant defect rework can extend Class A and B engagements beyond the upper bound and will be flagged in the QA status report.

10. Quality Assurance Governance

Clear ownership prevents quality gaps from falling between teams. The following structure defines who is responsible for each part of the quality process.

Role	Core responsibilities
QA Engineer	Test planning, test design and execution, defect reporting, regression and release validation, documentation, risk analysis, and automation planning.
Developers	Unit testing, defect resolution, technical support during testing, and deployment support.
Product / Business	Requirement clarification, business validation, UAT participation, and workflow verification.
Management / Leadership	Quality oversight, release approval, and risk-acceptance decisions.

11. Resourcing and Staffing Model

Governance (Section 10) defines who is responsible for each part of the quality process; this section defines how many of each role the framework actually requires. Without explicit resourcing, the SLA timelines in Section 9 silently depend on a single engineer holding multiple parallel engagements — a position that does not survive an audit, a sick week, or any meaningful product overlap.

11.1 Why a Single-Engineer Plan Fails

A Class A engagement runs 19 to 27 calendar days end-to-end. The moment two Class A initiatives overlap — for example, a wallet refactor concurrent with the loan launch — one engineer cannot hold both SLAs without breaching at least one. Class B and Class C work continues to arrive in parallel and competes for the same bandwidth. The risk is not theoretical; it is the predictable consequence of the SLAs themselves.

11.2 Throughput Analysis

At one full-time QA engineer dedicated to active testing, indicative annual throughput is bounded by the SLAs published in Section 9:

Class	Average duration	Theoretical annual max (1 engineer, dedicated)
A — High	~22 days	~10 – 12 projects per year (no overlap)
B — Medium	~9 days	~25 projects per year (no overlap)
C — Low	~2.5 days	High volume; fits in parallel with A/B work

Real-world capacity is materially lower once interruptions, retesting cycles, defect rework, automation maintenance, documentation, and non-project responsibilities (release support, on-call investigation, audit prep) are accounted for. The framework therefore plans on roughly 60% of theoretical maximum as usable capacity.

11.3 Headcount Plan by Business Phase

Phase	Composition	Rationale
Today (current scale)	1 × QA Lead, 1 × QA Engineer (minimum)	Two-person team eliminates the single-point-of-failure risk; permits one Class A and parallel Class B work without breaching SLA.
Pre-loan launch	1 × QA Lead, 2 × QA Engineers (1 dedicated to automation)	Adds a dedicated automation engineer to build the regression suite ahead of the loan launch, when manual-only coverage becomes untenable.
Loan launch and beyond	1 × QA Lead, 3 × QA Engineers (1 dedicated to loan, 1 to automation, 1 to ongoing product)	Dedicated loan QA owner ensures the higher-rigor requirements in Section 23 are sustained; existing engineers continue to cover wallet, wealth, and

Phase	Composition	Rationale
		investment products.

11.4 Skills Mix

The roles above are not interchangeable. The team must collectively cover functional testing, API and automation engineering, security awareness sufficient to coordinate with external pen-test firms, and enough financial-domain literacy to validate interest, ROI, and repayment calculations independently. The QA Lead role additionally carries planning, governance, and stakeholder reporting responsibilities.

11.5 Headcount Cost

Indicative fully-loaded annual cost is captured in Section 35 (Budget). Actual figures are subject to confirmation by HR and Finance against the prevailing Nigerian hiring market; the ranges in Section 35 are planning estimates, not approved compensation bands.

12. Testing Lifecycle Strategy

The Rosabon QA lifecycle follows a structured, repeatable validation flow. Each stage produces evidence that feeds the next and supports audit readiness.

Requirement Analysis → Test Planning → Test Case Design → Environment Preparation → Test Execution → Defect Reporting → Retesting → Regression Testing → UAT → Release Sign-off

13. Integrated QA Workflow and CI/CD Pipeline

The lifecycle in Section 12 describes the logical phases of testing. This section describes how those phases are mechanized into a continuous, automated quality pipeline that combines shift-left API testing, shift-right cross-device validation, and centralized reporting. Together, they move QA from siloed manual testing to a modern Quality Engineering operation.

13.1 The Five-Stage Automated Workflow

1	**Code commit & deploy** Code is checked into repository branches, triggering automated builds to the target test environment.
2	**Shift-left API test execution** Postman triggers automated validation collections against backend endpoints to ensure contract parameters, schemas, and security headers are intact.
3	**Shift-right & cross-device UI testing** TestMu AI / LambdaTest simultaneously boots real-device clouds to run automated functional and visual-regression suites across mobile viewports and browsers.
4	**Result aggregation & tracking** Execution telemetry from both automation engines and manual functional checklists is pulled directly into TestRail.
5	**Jira traceability linkage** TestRail maps test outcomes back to Jira tickets, automatically updating coverage logs and generating live metrics for stakeholders.

13.2 Why This Matters to the Business

Shift-left catches integration defects before they reach the UI, when they are cheapest to fix. **Shift-right** validates real customer experience on real devices, eliminating simulator false positives. **Centralized reporting** gives leadership a live, auditable view of release quality without chasing engineers for status. The net effect: faster releases, fewer escaped defects, and stronger audit posture.

14. Functional Testing Strategy

Functional testing validates that every feature behaves according to its business requirements. Coverage spans user onboarding, authentication, wallet funding, investment creation, transaction processing, notifications, profile management, and loan workflows. The objective is to verify expected behaviour, validate end-to-end user journeys, detect functional defects early, and confirm the correctness of underlying business logic.

15. Regression Testing Strategy

Regression testing ensures that new changes never break existing functionality. It covers authentication flows, wallet operations, investment workflows, transaction history, API integrations, and notifications. **Regression testing is mandatory before every production deployment for Class A and Class B projects.**

16. API Testing Strategy

The Rosabon ecosystem relies on APIs for service communication and financial operations, making API quality foundational to the platform. API testing validates responses and request integrity, verifies authentication, confirms transaction processing, ensures data consistency, and detects integration failures across wallet, investment, authentication, loan, notification, and third-party APIs.

Recommended tool: Postman (Professional plan) — see Section 34 for the full tooling stack.

17. Security Testing Strategy

Security testing protects customer accounts, financial assets, and sensitive information. Validation covers authentication, password policies, OTP handling, session management, access permissions, role-based access control, secure data transmission, and prevention of unauthorized access.

Standards reference: OWASP security principles, with emphasis on the OWASP Mobile and API Security Top 10.

18. Performance and Load Testing Strategy

Performance testing ensures the system remains stable and responsive under both expected and peak load. The goals are to validate responsiveness, detect bottlenecks, measure stability under sustained load, and confirm scalability readiness.

Key scenarios: concurrent wallet funding, simultaneous investment transactions, high login traffic, and loan-application spikes.

Recommended tool: Apache JMeter.

19. Mobile Application Testing Strategy

As a mobile-first fintech platform, the Rosabon App requires extensive device-, network-, and stability-focused testing. Real-device cloud testing (Section 34) eliminates simulator false positives and ensures visual consistency across viewports.

Focus area	What is validated
Device compatibility	Android and iOS, multiple screen sizes, and a range of OS versions
Network conditions	Weak connectivity, network switching, offline interruptions, and slow responses
Stability	Crash detection, memory usage, responsiveness, and background/foreground behaviour

Recommended monitoring: Firebase Crashlytics and Sentry.

20. Wallet System Testing Framework

The wallet is the most critical financial component in the ecosystem. Its testing framework combines functional coverage, strict integrity principles, and deliberate edge-case validation. All wallet-related changes are automatically classified as Class A (Section 8) and follow the corresponding SLA.

20.1 Core Testing Areas

Wallet funding, debit and credit operations, balance synchronization, transaction history, failed-transaction handling, duplicate prevention, and timeout handling.

20.2 Non-Negotiable Integrity Principles

Transactions must never disappear
Balances must remain accurate at all times
Every transaction must be fully traceable
Failures must be safely recoverable

20.3 Edge-Case Testing

Duplicate requests, network interruption mid-transaction, insufficient balance, delayed transaction responses, and reversal processing.

21. Wealth Management Testing Framework

The wealth management system requires validation of both functionality and financial accuracy. Functional coverage includes investment onboarding, funding, maturity tracking, withdrawals, investment history, and portfolio calculations.

Financial validation focuses on interest calculations, ROI accuracy, percentage calculations, maturity computations, and rounding behaviour — each verified against an independent expected-value source.

22. Investment Product Testing Framework

Each investment product undergoes dedicated testing against its own business rules.

Product	Testing coverage
RWIN	Product enrolment, funding validation, return calculations, withdrawal behaviour, eligibility checks
REAP	Investment creation, portfolio visibility, transaction validation, maturity handling
RTN	Product-logic validation, financial computations, wallet interaction, user transaction handling

23. Loan Management Testing Framework

Although the loan module is still in development, QA planning begins now so that validation assets are ready before launch. Planned coverage includes the loan application flow, eligibility verification, underwriting workflows, repayment-schedule calculations, disbursement, penalty handling, and repayment tracking.

Highest-risk areas: interest calculations, fraud prevention, repayment accuracy, and customer-data validation.

23.1 Mandatory Pre-Launch Gates (Hard Stops)

The loan module presents existential legal and financial risk. Internal QA validation alone is insufficient to authorize launch. The following gates are non-negotiable and must be cleared in writing before the first customer is offered credit. Failure to clear any gate is a hard stop.

#	Mandatory pre-launch gate	Owner
1	Independent third-party validation of the interest calculation engine and repayment-schedule generator against an external financial model. Internal expected-value tests are necessary but not sufficient.	**External actuarial / audit firm**
2	External application security review (penetration test) of the loan module by an accredited firm. All Critical and High findings must be remediated and re-tested before launch.	**External security firm**
3	Regulatory compliance attestation by Legal and Compliance, confirming the module satisfies applicable CBN lending guidelines, SEC obligations (where investment-linked), NDPA/GAID data-protection requirements, and AML/KYC controls (Section 33).	**Legal & Compliance**
4	Operational readiness sign-off covering fraud detection, collections workflow, dispute handling, and customer support runbooks.	**Operations & Risk**
5	Rollback drill executed on staging (not merely documented): a complete loan-disbursement rollback exercised end-to-end within the past 30 days.	**QA & DevOps jointly**

Recurring obligation: Gate 1 (calculation validation) and Gate 2 (external security review) are repeated annually thereafter, and on any material change to lending parameters or the underwriting engine.

24. User Acceptance Testing (UAT)

UAT confirms that business stakeholders validate workflows before production release. Its objectives are to validate business expectations, confirm operational readiness, ensure customer-workflow satisfaction, and secure formal release acceptance. Participants include the product team, business stakeholders, operations representatives, and QA. UAT is mandatory for Class A and Class B initiatives.

25. Environment Management Strategy

Reliable testing depends on properly controlled, isolated environments.

Environment	Purpose
Development	Active development and early testing
Staging / UAT	Release validation and business testing
Production	Live, customer-facing environment

Environment requirements: stable test data, controlled deployments, environment isolation, and access management.

26. Defect Management Process

Defect management ensures every issue is tracked, prioritized, and resolved consistently.

26.1 Defect Lifecycle

Identified → Logged → Assigned → Fixed → Retested → Closed

26.2 Severity & Priority Levels

Severity	Description
P1 — CRITICAL	System blocker, transaction-loop failure, data exposure, or authentication crash
P2 — HIGH	Major functional defect without an immediate workaround for the user
P3 — MEDIUM	Limited functional issue with a standard administrative workaround
P4 — LOW	Minor visual misalignment or copy modification request

26.3 Required Fields in Every Defect Report

Title; description; reproduction steps; expected result; actual result; environment; screenshots or logs; severity; and priority.

26.4 Production Incident Feedback Loop

Defect leakage (Section 30) is measured but does not close itself. Every P1 or P2 defect that reaches production triggers a structured feedback loop so the same class of escape cannot recur unnoticed.

Trigger	Required action	Output
P1 or P2 defect in production	Blameless postmortem within 5 business days, attended by QA, Engineering, and Product	A new regression test that would have caught the issue, added to the appropriate automated suite
Pattern of similar escapes	Root-cause review against the risk register and process improvement filed in Section 32 (Risk Management)	Updated risk score and, if needed, revised testing rigor for affected modules
Any release requiring rollback	Rollback exercised end-to-end on staging; lessons captured in the release retrospective	Hardened rollback runbook and a tested recovery path for the next release
Quarterly cadence	Rollback drill on the staging environment regardless of incident history	Live evidence that recovery works; not just a documented plan

Principle: a documented rollback plan is not a tested rollback plan. The QA function owns ensuring the difference is closed quarterly.

27. Automation Testing Strategy

Automation underpins scalability, efficiency, and regression stability. The objectives are to reduce repetitive manual effort, accelerate regression, increase release confidence, and enable CI/CD integration. Automation is introduced in deliberate phases so that it strengthens — rather than destabilizes — delivery.

Phase	Focus
Phase 1	Stabilize and document manual testing processes
Phase 2	Implement API automation (Postman collections, scheduled monitors)
Phase 3	Automate critical regression suites and cross-device flows
Phase 4	Integrate automated testing into the CI/CD pipeline with telemetry into TestRail

Tooling: see Section 34 for the full Quality Engineering tooling stack.

28. Project QA Execution Checklist

This checklist standardizes how every project is taken from intake to production release. It is mandatory for Class A and Class B initiatives and may be lightly applied to Class C.

28.1 Phase 1 — Requirements Ingestion & Shift-Left Initialization

☐	**PRD inspection**	Review the PRD or RFP to isolate automated dependencies, operational edge cases, and compliance criteria.
☐	**Project classification flagging**	Classify the initiative as Class A, B, or C to set testing scope and SLA automatically.
☐	**Risk surface mapping**	Complete the risk assessment matrix for the project, identifying high-vulnerability components (identity syncing, accounting calculations, multi-tenant boundaries).
☐	**Central repository setup**	Spin up a structured workspace in the test management tool: 01_Test_Plans / 02_Test_Cases / 03_Automation_Suites / 04_Execution_Logs.

28.2 Phase 2 — Test Case Design & Tooling Configuration

☐	**Traceability structuring**	Author test steps in the test management tool, mapping every test script to its corresponding Jira user story ID.
☐	**Postman collection construction**	Build API verification scripts validating token handling, schemas, HTTP response codes, and error models.
☐	**Cross-device target selection**	Select device matrices in the testing cloud — screen viewports, Android tiers, and iOS builds — mapped to the customer audience footprint.
☐	**Test data seeding**	Stage test data files or automated seeding queries in sandbox environments. Live customer profiles are never used.

28.3 Phase 3 — Active Testing Execution & Defect Triage

☐	**Backend endpoint verification**	Execute the Postman automation suite against test targets to verify endpoint functionality before testing UI elements.
☐	**Frontend real-device testing**	Run interface verification across the device cloud — responsiveness, offline cache, navigation loops.
☐	**Defect logging enforcement**	Document failed steps in the issue tracker with step-by-step reproduction, system state, visual logs, and backend exception payloads.
☐	**Triage prioritization routing**	Route each defect using P1 (critical) / P2 (high) / P3 (medium) / P4 (low) per Section 26.
☐	**Regression verification**	Execute targeted regression on fixed deployments to confirm the fix

		works and has not impacted adjacent features.

28.4 Phase 4 — Full Scope Regression & Final Gate Validation

☐	**Master regression run**	For Class A and B projects, run the full automated regression suite to ensure broader ecosystem stability.
☐	**Compliance & security evaluation**	Run role-privilege validation checks to confirm user data isolation and verify encryption of data in transit.
☐	**UAT lifecycle oversight**	Coordinate UAT workflows with business stakeholders and track final feedback in a structured sign-off document.
☐	**Pre-flight release verification**	Confirm all P1/P2 defects are closed, automation passes meet baseline goals, and rollback plans are in place.
☐	**Central dashboard sync**	Upload final execution logs, test-run metrics, and stakeholder sign-offs to the tracking system to clear the project for production.

29. Release Readiness Framework

No production release may proceed until every item below is satisfied. This checklist is the final quality gate and is owned jointly by QA and Management.

☐	Smoke tests passed
☐	Critical (P1) and High (P2) defects resolved
☐	Regression testing completed
☐	Wallet transactions validated
☐	Investment calculations verified
☐	API responses validated
☐	UAT sign-off completed
☐	Rollback plan documented
☐	Release approval granted

30. Reporting and Metrics Framework

QA reporting provides leadership with visibility into quality health and release confidence. Standard reports include test-execution, defect, regression, release-readiness, and risk-assessment reports.

30.1 Baseline-First Measurement

Targets without baselines cannot be defended. Phase 1 of the implementation roadmap (Section 36) explicitly measures the current state for each metric below. Year-1 targets are then ratified by leadership against the measured baseline, not asserted in the abstract. The figures in the next table are planning targets — sensible end-of-Year-1 outcomes for a fintech of comparable scale — to be confirmed once the baseline is captured.

30.2 Key Quality Metrics, Baseline, and Year-1 Targets

Metric	Definition	Baseline	Year-1 target
Test pass rate	% of executed test cases passing before release	TBD	**≥ 95%**
Defect leakage	Defects found in production vs. total found	TBD	**≤ 5%**
Critical-defect escape	Critical (P1) defects reaching production	TBD	**0**
Regression coverage	Critical workflows covered by regression	TBD	**100%**
Defect resolution time	Median time to resolve critical (P1) defects	TBD	**≤ 24 hrs**
Crash-free sessions	Mobile sessions without a crash	TBD	**≥ 99.5%**
Release stability	Releases shipped without rollback / hotfix	TBD	**≥ 95%**

"TBD" denotes a metric that will be measured during Phase 1 (Discovery and Assessment). The first quarterly QA report after Phase 1 will replace these with measured values and ratified targets.

31. Documentation Standards

Standardized documentation underpins traceability and audit readiness. Required artifacts include test plans, test cases, bug reports, QA reports, release sign-off reports, risk reports, and UAT reports. Maintaining these consistently improves traceability, supports audits, strengthens collaboration, and preserves a reliable historical record.

32. Risk Management Framework

Proactive risk management identifies and mitigates quality threats before they reach customers.

Major fintech risk	Primary mitigation
Financial miscalculations	Independent expected-value validation and financial regression suites
Unauthorized transactions	Security testing, RBAC validation, and access monitoring
Security vulnerabilities	OWASP-aligned testing and secure-transmission checks
Data inconsistencies	Balance-synchronization and data-integrity validation
Transaction duplication	Duplicate-prevention and idempotency testing
Service downtime	Performance/load testing and production monitoring

33. Compliance and Audit Readiness

Important context: compliance is not security, and OWASP is not a regulator. This section names the Nigerian regulatory perimeter that applies to a fintech holding customer funds, running investment products, and (shortly) lending. QA's role is to support evidence and testing for these obligations; the regulatory perimeter itself must be co-signed by Legal and Compliance before this document is presented externally. The list below is QA's working assumption, not legal advice.

33.1 Applicable Regulatory Perimeter

Regulator / regime	Key obligations relevant to QA evidence
CBN — Central Bank of Nigeria	Principal regulator of financial services under the Banks and Other Financial Institutions Act. Sets licence conditions, capital and compliance thresholds (raised in 2025), AML/CFT regulations (including the 2025 automation mandate), and supervisory inspections. QA evidence: transaction logging, controls testing, audit trail retention.
NDPA 2023 + GAID 2025 — Nigeria Data Protection Act and General Application and Implementation Directive (effective Sept 2025), enforced by the NDPC	Rosabon, as a fintech processing large volumes of customer data, is very likely classified as a Data Controller / Processor of Major Importance (DCPMI). Obligations include: NDPC registration, annual compliance audit (filed with NDPC), 72-hour breach notification, privacy policy alignment, data-subject rights handling, and risk-based periodic compliance audits. QA evidence: breach-detection paths tested, consent flows tested, data-subject request flows tested.
SEC Nigeria — under the Investments and Securities Act (ISA) 2025	Applies where RWIN, REAP, or RTN qualify as regulated investment schemes. Obligations cover offering disclosures, suitability, and fair-dealing. QA evidence: accurate disclosure rendering, returns calculations, fee transparency.
NFIU + EFCC + SCUML — AML/CFT enforcement chain	Suspicious Transaction Reports filed to the NFIU; investigations led by EFCC. Nigeria exited the FATF grey list in October 2025; standards remain stringent. QA evidence: STR-trigger logic tested, customer due diligence flows tested, sanctions-screening tested.
BVN / NIN identity harmonization	Identity verification expectations require integrated BVN and NIN checks for onboarding and high-risk transactions. QA evidence: identity-verification flow tested across success and failure paths.

33.2 QA's Specific Compliance Responsibilities

QA does not own regulatory interpretation, but does own the evidence that controls actually work. The function is accountable for:

- Test-evidence retention aligned with audit cycles (annual NDPC, CBN inspection-on-demand).
- End-to-end test coverage of customer onboarding and KYC, sanctions screening, and AML transaction-monitoring triggers.

- Validated breach-detection and notification paths capable of meeting the 72-hour NDPC reporting window.
- Data subject access, rectification, and deletion flows tested against NDPA requirements.
- Audit trail completeness for every customer-money-moving transaction.

33.3 Required Next Step — Legal Co-Sign

> ***This compliance perimeter is QA's planning view. Before this framework is treated as a complete regulatory map, it must be reviewed and co-signed by Legal and Compliance, and any additional licences, circulars, or directives specific to Rosabon's current operating permissions must be added. QA will not self-attest regulatory coverage.***

34. QA Tooling Stack

The Quality Engineering pipeline is built on three integrated pillars: a backend engine for API testing, a real-device cloud for cross-platform UI validation, and a command centre for centralized reporting. Together they close the gap between backend logic, customer-facing client interfaces, and compliance reporting.

34.1 The Three-Pillar Stack

BACKEND ENGINE **Postman Professional**	CROSS-DEVICE CLOUD **TestMu AI / LambdaTest**	COMMAND CENTRE **TestRail Professional**
Shift-left API testing. Builds, versions, and automates API test collections. Scheduled API monitoring alerts catch broken integration contracts before they reach downstream environments. Postbot AI accelerates script generation.	**Real-device execution fabric.** Cloud-based real iOS and Android hardware plus browser containers. Eliminates simulator false positives. Ensures visual regression consistency and parallelizes cross-browser suites.	**Single source of truth.** Ingests execution data from Postman, TestMu, and manual scripts. Dynamic compliance dashboard for leadership. Maps every metric back to Jira user stories.

34.2 Supporting Tools

Category	Tool
Performance / load testing	Apache JMeter
CI/CD orchestration	GitHub Actions / Jenkins
Mobile crash monitoring	Firebase Crashlytics
Application monitoring	Sentry
Issue tracking	Jira (integrated with TestRail)

35. Budget and Investment Plan

This is the fully-loaded view of what the framework actually costs to operate, not just the tooling line. Tooling licences are the cheapest part; people, external penetration testing, and a staging environment that mirrors production are the genuine costs that determine whether the strategy holds in practice. All figures are planning estimates in Nigerian Naira. Foreign-currency-priced services (Postman, TestRail, TestMu, and most external pen-testing firms) are converted at an indicative planning rate of ₦1,400 / USD; final amounts will follow the prevailing rate on each invoice date. Actuals for people will be confirmed by HR and Finance.

35.1 Cost Categories

Category	Components	Annual (₦, indicative)
Tooling licences	Postman Professional (2 seats), TestRail Professional (1 seat), TestMu real-device cloud (capped at 1 parallel session)	≈ ₦2.3M
People	Phased headcount per Section 11 — fully-loaded cost (salary + statutory benefits + on-costs). See breakdown in 35.2.	₦29M – ₦85M
External penetration testing	Annual application security review + per-release review for the loan module (mandatory under Section 23, Gate 2)	₦14M – ₦35M
Performance / load infrastructure	Cloud-based load generation, expert-led test design, and execution (JMeter is free; the environment and expertise are not)	₦4M – ₦11M
Production-mirror staging environment	Staging that genuinely mirrors a financial production system. To be costed by Engineering and DevOps. Placeholder included for visibility.	₦4M – ₦14M

> **Indicative fully-loaded total: roughly ₦53 million to ₦147 million per year, depending on phase, with the lower bound reflecting today's two-person team and the upper bound reflecting post-loan-launch scale. Tooling licences are one line item among five; people, external pen-testing, performance infrastructure, and staging together drive the majority of the total.**

35.2 Headcount Cost Bands by Phase

Phase (per Section 11)	Team	Annual cost (₦, indicative)
Today — current scale	1 Lead + 1 Engineer	₦29M – ₦52M
Pre-loan launch	1 Lead + 2 Engineers	₦38M – ₦69M
Loan launch and beyond	1 Lead + 3 Engineers	₦46M – ₦85M

Ranges reflect the prevailing Nigerian QA hiring market for mid- and senior-level engineers as a planning input. Actual compensation bands are owned by HR.

35.3 Tooling Detail (Capped)

Tool	Plan	Seats	Monthly (₦)	Annual (₦)
Postman	Professional	2	₦41K / seat	**₦975K**
TestRail	Professional	1	₦53K / seat	**₦640K**
TestMu AI	Real Device — 1 parallel session (capped)	1	≈ ₦55K – ₦59K	**≤ ₦700K**
Total tooling licences (capped)				**≈ ₦2.3M**

Note on FX exposure: Postman, TestRail, and TestMu are billed in USD by their vendors. ₦ figures use the ₦1,400 / USD planning rate; actual invoice amounts will follow the prevailing rate on each billing date. A significant naira depreciation against the dollar would increase the tooling line and should be reviewed at the quarterly cadence in Section 35.6.

Cap rationale: the TestMu service is sold on a usage basis that can otherwise be open-ended. It is therefore capped at a single parallel session with an explicit annual ceiling. Any need to exceed this cap returns to leadership for approval rather than appearing as a budget overrun.

35.4 What the Investment Buys

- **A two-to-four-person QA team** sized to the SLAs in Section 9, eliminating the single-engineer dependency.
- **Automated API validation** with scheduled monitors that surface integration regressions before customers see them.
- **Real-device cross-platform testing** on actual iOS and Android hardware, eliminating the false confidence of simulator-only testing.
- **A centralized quality dashboard** that maps every test result back to Jira tickets and gives leadership audit-ready evidence on demand.
- **Independent external security and calculation review** for the wallet engine and the loan module, satisfying the mandatory gates in Sections 23 and 33.
- **A defensible compliance posture** with NDPA/NDPC, CBN, and SEC obligations covered by tested controls and retained evidence.

35.5 What This Budget Does Not Yet Cover

Honest scoping requires naming the items left out so leadership is not surprised later:

- Production observability beyond Crashlytics free tier and Sentry's lower plans.
- Certifications, training, and conference attendance for the QA team.
- Bug-bounty programme (recommended once the platform is more mature).
- Contingency for emergency third-party engagements (incident response retainers).

35.6 Scaling and Review Cadence

> *This budget is reviewed quarterly alongside the metrics in Section 30. Material variance (>15%) is escalated to leadership. As the loan module launches, the upper bounds of each category should be expected, not the lower.*

36. Implementation Roadmap

The framework is adopted progressively across four phases, each building on the last.

Phase	Key activities
1 — Discovery & Assessment	Understand architecture, identify critical workflows, audit current QA maturity, review release processes
2 — Foundation Setup	Define testing processes, create documentation templates, establish reporting standards and release procedures; procure initial tooling licenses
3 — Structured Operations	Implement regression processes, expand API testing, improve defect governance, apply project classification across the pipeline
4 — Automation & Optimization	Introduce automation frameworks, integrate CI/CD testing, expand monitoring and metrics across the full Quality Engineering pipeline

37. Immediate Action Plan

The following priorities establish momentum in the first weeks of adoption, sequenced so that headcount, compliance sign-off, and budget approval are in place before downstream activities depend on them.

1. **Headcount approval —** approve at minimum the two-person team per Section 11.3 to eliminate the single-engineer dependency before any further Class A initiatives are accepted.
2. **Legal & Compliance co-sign —** Legal reviews Section 33 and ratifies the regulatory perimeter; any Rosabon-specific licences or directives are added. The framework is not externally distributable until this is complete.
3. **Budget approval —** approve the fully-loaded budget in Section 35 with the TestMu cap applied, and authorise procurement to source external-penetration-testing quotations.
4. **Phase 1 baseline measurement —** complete the baseline measurement described in Section 30 within the first six weeks so Year-1 targets are ratified against real data.
5. **Product, architecture & process audit —** complete the standard Phase-1 activities: review modules and workflows, understand backend integrations and deployment, evaluate current defect tracking and release-validation.
6. **Tooling procurement —** provision Postman, TestRail, and TestMu licences per Section 35 once headcount and budget approvals are in hand.
7. **Framework adoption —** introduce documentation standards, establish governance, and begin structured reporting.

38. Conclusion

The Rosabon App ecosystem operates in a high-risk fintech environment where quality assurance must extend well beyond traditional software testing. This framework establishes a structured strategy centred on financial accuracy, system reliability, security assurance, transaction integrity, release confidence, risk reduction, and operational continuity.

By implementing it, Rosabon Financial Services strengthens product stability, customer trust, operational efficiency, engineering maturity, and release governance. The integrated tooling stack and modest initial investment create a Quality Engineering pipeline that is automated, auditable, and ready to scale alongside the business.

Equally important, the framework creates a foundation for future growth, automation adoption, and continuous quality improvement across the ecosystem.

> ***Quality is not the final gate before release — it is the discipline that earns customer trust on every transaction.***